# AEP System Engineering Backlog

**Traceability Hierarchy:** Requirements (REQ) → Capabilities (CAP) → Features (FEAT) → Tasks (TASK) → Functions (FUNC) → Acceptance Criteria (AC)
**Version:** v1.1 · **Date:** 2026-11-09 · **Architecture Baseline:** `aep-hld-v1.1.html` & `aep-lld-v1.1.html` · **PR:** #133

> **Architecture & Traceability Standard:** This backlog formalizes the architectural decisions and low-level design specifications of the Autonomous Engineering Platform (AEP). Every work item is broken down into concrete Go packages, data schemas, CLI surface, IPC sockets, and strictly verifiable acceptance criteria (Gherkin/testable conditions).

## Table of Requirement Domains

| ID | Requirement Domain | Core Capabilities | LLD Tasks Mapped | Status |
|---|---|---|---|---|
| **REQ-01** | Dual-Lane Sandboxed Execution Runtime | CAP-01 (Tier-1 Ralph Container Isolation) & CAP-02 (Tier-2 Local-First Sandbox Factory) | T0, T6, T9, T10, T11 | IN PLAN |
| **REQ-02** | Substrate Core Services & Secure Relays | CAP-03 (NeuroStrata Memory IPC Relay) & CAP-04 (GHG Gateway Credential Guard) | T1b, T3, T9 | IN PLAN |
| **REQ-03** | Unified Ledger & Task Governance | CAP-05 (Beads Single Source of Truth) & CAP-06 (Deterministic Dispatch Engine) | T5, T7, T8 | IN PLAN |
| **REQ-04** | Multi-Hardware Scalable Resource Profiles | CAP-07 (Hardware Profile Presets) & CAP-08 (Dual-Constraint Admission Control) | T1, T4 | IN PLAN |
| **REQ-05** | Policy Enforcement & Platform Boot Gating | CAP-09 (Cryptographic HEAD Tamper Verification) & CAP-10 (Full Parity Boot Gate) | T2, T3 | IN PLAN |
| **REQ-06** | Verification, Quarantine & Audit Trail | CAP-11 (Isolated Re-Verification Sandbox) & CAP-12 (Cryptographic Receipts) | T7, T10, T11 | IN PLAN |

## Domain 1: REQ-01 Dual-Lane Sandboxed Execution Runtime

The platform must govern two distinct execution lanes: high-velocity inner-loop container lanes (Tier 1: Ralph) and strongly isolated, untrusted code pipelines (Tier 2: ESF Factory) with a local-first sandbox hierarchy.

CAP-01 **Tier-1 Ralph-Loop Lane Container Orchestration** REQ-01.1 · Package: `internal/lane`

Executes parallel inner-loop code refinement tasks in isolated OCI containers with per-lane git worktrees and zero direct external network egress.

FEAT-01.1 **ZERO-EGRESS OCI CONTAINER & WORKTREE ISOLATION**

Spawns ephemeral containers with `--network none`, read-only rootfs, tmpfs `/tmp`, virtiofs-mounted git worktree, and mounted IPC socket relays.

TASK-01.1.1 **IMPLEMENT LANE MANAGER & CONTAINER PROCESS DRIVER** LLD T6

Create `internal/lane` driver managing container spawning, worktree allocation, fence files, and graceful cleanup.

**Functions & Interfaces:**

```
type Manager interface {
  Claim(ctx context.Context, issueID string, tier Tier) (*Manifest, error)
  Run(ctx context.Context, laneID string, spec RunSpec) (*Receipt, error)
  Release(ctx context.Context, laneID string) error
  Reap(ctx context.Context, dryRun bool) ([]ReapRecord, error)
}
```

**Acceptance Criteria (AC-01.1.1):**

- **AC-01.1.1.1 (Network Fence):** GIVEN a spawned lane container, WHEN a process inside executes `curl https://api.github.com` or any network socket call, THEN the OS returns `ENETUNREACH` or `EPERM` immediately.
- **AC-01.1.1.2 (Filesystem Immutability):** GIVEN a lane container, WHEN a process attempts to write outside `/workspace` or `/tmp`, THEN the kernel returns `EROFS` (Read-only file system).
- **AC-01.1.1.3 (Worktree Boundary):** GIVEN lane `t1-7f3a9c` operating on issue `aep-142`, WHEN git commands run inside, THEN git mutations occur strictly within `$AEP_WORKTREES_ROOT/aep-142` without modifying the parent repository `develop` worktree.
- **AC-01.1.1.4 (Crash Fence & Reaping):** GIVEN a launcher process killed with `SIGKILL` while the container lives, WHEN `aep lane reap` runs, THEN it marks the lane active and refuses to reap until the container terminates.

CAP-02 **Tier-2 Local-First Sandbox Factory Orchestration**

REQ-01.2 · Package: `internal/factory`

Executes untrusted dependencies, multi-repo changes, and CVE remediation pipelines in hardware microVMs or user-space kernels before admitting patches to host review.

FEAT-02.1 **PLUGGABLE LOCAL-FIRST SANDBOX PROVIDER INTERFACE**

Abstracts sandbox execution across `cube-kvm` (Linux KVM), `libkrun` (macOS Apple Silicon `vz`), and `gvisor` (`runsc systrap` for KVM-free hosts). Designates remote clusters strictly as an emergency fallback.

TASK-02.1.1 **BUILD FACTORY ADAPTER & SANDBOX BACKEND RESOLVER** LLD T10

Implement `internal/factory` wrapping the ESF runtime with automated backend probing and egress guard validation.

**Functions & Interfaces:**

```
type SandboxBackend string
const (
  BackendCubeKVM SandboxBackend = "cube-kvm"
  BackendLibkrun SandboxBackend = "libkrun"
  BackendGVisor SandboxBackend = "gvisor"
  BackendRemote SandboxBackend = "remote-api" // Emergency fallback of last resort
)
type Client interface {
  Doctor(ctx context.Context) (*DoctorReport, error)
  ResolveBackend(ctx context.Context) (SandboxBackend, error)
  Run(ctx context.Context, req RunRequest) (RunHandle, error)
  VerifyEgressProbe(ctx context.Context, runID string) (bool, error)
}
```

**Acceptance Criteria (AC-02.1.1):**

- **AC-02.1.1.1 (Local Hierarchy):** GIVEN boot on Linux with `/dev/kvm`, THEN `ResolveBackend` returns `cube-kvm`; GIVEN macOS with `vz`, THEN returns `libkrun`; GIVEN a host without KVM (Intel N100 or non-bare-metal cloud VM), THEN returns `gvisor`.
- **AC-02.1.1.2 (Emergency Fallback Restraint):** GIVEN `BackendRemote`, WHEN local backends are operational, THEN `BackendRemote` is NEVER selected. Remote is selected if and only if all local backends fail AND remote cluster credentials are explicitly configured in `factory.toml`.
- **AC-02.1.1.3 (Egress Guard):** GIVEN `factory.toml` configured with `hardening.acknowledge_open_egress = true`, WHEN `Doctor` or `Run` is called, THEN execution is rejected with error `ErrGovernanceViolation: open egress posture acknowledged`.
- **AC-02.1.1.4 (Probe Cryptographic Verification):** GIVEN a completed factory pipeline run, WHEN `VerifyEgressProbe` inspects the VM evidence bundle, THEN it verifies the SHA-256 digest emitted by the in-VM probe. If probe failed or unverified, PR generation is blocked.

## Domain 2: REQ-02 Substrate Core Services & Secure Relays

Core platform services (NeuroStrata cognitive memory and GHG rate-limited GitHub gateway) run on the substrate host and are exposed to zero-egress containers strictly through supervisor-mediated IPC socket relays.

CAP-03 **NeuroStrata Cognitive Long-Term Memory IPC Relay** REQ-02.1 · Package: `internal/memoryd`

Relays cognitive learnings, architecture decisions, and operational context between zero-egress containers and host NeuroStrata daemon.

### FEAT-03.1 HOST IPC SOCKET FORWARDING & COGNITIVE INSCRIPTION

Runs `aep memoryd` listening on a UNIX domain socket ( `/run/aep/lanes/<laneId>/memory.sock` ), forwarding JSON-RPC requests to host `127.0.0.1:34343` .

#### TASK-03.1.1 BUILD `MEMORYD` HOST SOCKET RELAY DAEMON LLD T3

Implement the memory relay daemon with request rate limiting, schema validation, and health probing.

**Functions & Interfaces:**

```
type RelayServer struct {
  SocketPath string
  UpstreamTCP string // default: 127.0.0.1:34343
}
func (s *RelayServer) Start(ctx context.Context) error
func (s *RelayServer) ProbeStatus(ctx context.Context) (*MemorydStatus, error)
func InscribeReceipt(ctx context.Context, client MemoryClient, r Receipt) error
```

**Acceptance Criteria (AC-03.1.1):**

- **AC-03.1.1.1 (Loopback Isolation):** GIVEN lane containers running in isolated network namespaces ( `--network none` ), WHEN communicating over `memory.sock` , THEN memory operations succeed without container access to host network loopback.
- **AC-03.1.1.2 (Substrate Down Resilience):** GIVEN host NeuroStrata daemon is offline, WHEN `memoryd.ProbeStatus` runs during `aep boot` , THEN boot fails closed with exit code 1 (preventing unmonitored lane starts).
- **AC-03.1.1.3 (Inscription on Close):** GIVEN a completed task, WHEN `InscribeReceipt` executes, THEN task summary, failure modes, and code diff hash are recorded into the persistent vector memory table.

CAP-04 **Rate-Limited GitHub Gateway Relay (`ghg`)** REQ-02.2 · Package: `internal/gateway`

Acts as an authenticated proxy for GitHub operations, isolating personal access tokens (PAT) on the host and coalescing API requests.

FEAT-04.1 **CREDENTIAL GUARD & REQUEST POLLER COALESCING**

Exposes `/run/aep/lanes/<laneId>/ghg.sock` to lanes, proxying requests to `tools/ghg/bin/ghg` while keeping secrets out of container memory.

TASK-04.1.1 **IMPLEMENT `AEP GWD` DAEMON & LANE CLIENT** LLD T1B

Construct UNIX socket listener parsing JSON commands ( `pr-status` , `pr-create` , `check-runs` ), executing host `ghg.py` , and returning sanitized results.

> **Functions & Interfaces:**
> ```
> type Request struct {
>  Op string `json:"op"` // e.g. "pr-view", "checks-status"
>  Args []string `json:"args"`
>  LaneID string `json:"lane_id"`
> }
> func ServeConn(conn net.Conn, runner GHGRunner, cache PollerCache) error
> func CoalescePRStatus(ctx context.Context, pr int) (*PRStatus, error)
> ```

**Acceptance Criteria (AC-04.1.1):**

- **AC-04.1.1.1 (Token Zero-Leakage):** GIVEN lane containers, WHEN inspected via `docker inspect` , `env` , or memory core dump, THEN `GITHUB_TOKEN` or `GH_TOKEN` is completely absent.
- **AC-04.1.1.2 (Request Coalescing):** GIVEN 8 parallel lanes polling CI status for PR #133 within 30 seconds, THEN `gateway` executes upstream GitHub API calls exactly once and serves cached results to all 8 lanes.
- **AC-04.1.1.3 (Write Scoping):** GIVEN a lane requesting PR creation, THEN the gateway verifies that the target branch matches the lane's registered `issueId` manifest before signing and executing the call.

## Domain 3: REQ-03 Unified Ledger & Task Governance

The platform must maintain a unified, durable issue ledger (using Beads `bd` ) as the single source of truth, governed by deterministic tier dispatching and audit recording.

CAP-05 **Beads Issue Ledger & Dependency Resolution** REQ-03.1 · Package: `internal/ledger`

Queries local Beads database ( `bd list --json --all --limit 0` ), models issue states, and validates dependency DAGs.

FEAT-05.1 **DETERMINISTIC CANDIDATE SELECTION & TIER CLASSIFICATION**

Fetches unwindowed issue candidates, checks for terminal blocker states, and resolves tier tags ( `tier:local` vs `tier:factory` ).

TASK-05.1.1 **BUILD BEADS ADAPTER & ISSUE GRAPH PARSER** LLD T5

Create Go client executing `bd` CLI through injectable runner seam with typed error handling.

**Functions & Interfaces:**

```
type Issue struct {
  ID string `json:"id"`
  State string `json:"state"`
  TierTag Tier `json:"tier_tag"`
  Dependencies []string `json:"dependencies"`
}
func ListEligible(ctx context.Context, runner Runner) ([]Issue, error)
func ResolveBlockers(target Issue, all []Issue) (isBlocked bool, blockers []string)
```

**Acceptance Criteria (AC-05.1.1):**

- **AC-05.1.1.1 (Unpinned Limit Guard):** GIVEN candidate fetch, WHEN executing `bd list` , THEN `--limit 0` is explicitly passed; truncation raises `ErrFetchCeilingExceeded` .
- **AC-05.1.1.2 (Missing Tier Tag):** GIVEN an issue with no `tier:*` tag and no `risk:low` tag, WHEN candidate selection runs, THEN the issue is rejected from dispatch (cannot place unclassified work).
- **AC-05.1.1.3 (DAG Blockers):** GIVEN issue B depending on issue A, WHEN issue A is in state `open` or `in_progress` , THEN issue B is marked blocked and skipped.

CAP-06 **Deterministic Dual-Lane Dispatcher & Quarantine Governance** `REQ-03.2 · Package: internal/dispatch`

Evaluates candidate tasks against substrate health, runtime sandbox availability, and admission capacity; strictly enforces quarantine over silent downgrade.

FEAT-06.1 **PURE DECISION ENGINE & AUDIT TRAIL WRITER**

Implements the dispatch decision matrix (§7 of LLD), producing auditable JSON decision records for every admission, wait, or quarantine.

TASK-06.1.1 **IMPLEMENT DISPATCHER DECISION ENGINE** LLD T7

Develop pure `dispatch.Decide` function and CLI subcommand `aep dispatch --dry-run`.

**Functions & Interfaces:**

```
type DecisionAction string
const (
  ActionRunLocal DecisionAction = "run-local"
  ActionRunFactory DecisionAction = "run-factory"
  ActionWaitCapacity DecisionAction = "wait"
  ActionQuarantine DecisionAction = "quarantine"
)
func Decide(ctx context.Context, d Deps, issue Issue) DecisionRecord
func WriteDecision(record DecisionRecord) error
```

**Acceptance Criteria (AC-06.1.1):**

- **AC-06.1.1.1 (Quarantine on Sandbox Failure):** GIVEN an issue tagged `tier:factory`, WHEN all local sandboxes (Cube/KVM, libkrun, gVisor) are unavailable and no remote cluster is configured, THEN decision is `quarantine` with reason `blocked:unsupported-tier2-no-sandbox` (exit 4).
- **AC-06.1.1.2 (Prohibition of Silent Downgrade):** GIVEN a `factory` task on an unsupported host, WHEN operator has NOT provided `--override-tier1-risk-reassessment`, THEN the task is NEVER routed to Tier 1.
- **AC-06.1.1.3 (Audit Persistence):** GIVEN any dispatch evaluation, THEN a decision file is written to `$AEP_STATE_DIR/decisions/<ts>-<issue>.json` recording probed state, timestamp, and rationale.

## Domain 4: REQ-04 Scalable Multi-Hardware Profiles & Dynamic Admission Control

The platform must scale reliably across hardware profiles—from low-power 2 vCPU / 8 GB edge machines (Intel N100) up to 12 vCPU / 48 GB developer workstations (Apple M3 Max)—governed by dynamic admission control.

CAP-07 **Multi-Hardware Profile Registry & Probing Engine**

REQ-04.1 · Package: `internal/profile`

Detects host family ( `mac-colima-vz` vs `linux-native` ), CPU cores, RAM, and hardware virtualization capabilities, mapping to size presets.

FEAT-07.1 **PROFILE DETECTION CLI & PRESET VALIDATOR**

Provides `aep profile detect` and `aep profile show` , ensuring system reservations and container budgets do not overcommit host memory.

TASK-07.1.1 **BUILD PROFILE DETECTION & CONFIG PACKAGE** LLD T1

Implement hardware discovery probing `/proc/cpuinfo` , `/proc/meminfo` (or macOS `sysctl` ), and `/dev/kvm` .

**Functions & Interfaces:**

```
type Profile struct {
  Name string
  SubstrateMemMB int
  Tier1BudgetMB int
  Tier1LaneMax int
  Tier2BudgetMB int
  SandboxBackends []SandboxBackend
}
func DetectHost(ctx context.Context) (*Profile, error)
func LoadPreset(name string) (*Profile, error)
```

**Acceptance Criteria (AC-07.1.1):**

- **AC-07.1.1.1 (Nano-8G Support):** GIVEN host with 8 GB RAM (e.g. Intel N100), WHEN `aep profile detect` runs, THEN it selects `nano-8g` : Substrate 2.5 GB, Tier-1 4.0 GB (1–2 lanes), Tier-2 1.5 GB via `gvisor` .
- **AC-07.1.1.2 (Workstation-48G Support):** GIVEN Apple M3 Max with Colima `vz` and `--nested-virtualization` , THEN it selects `workstation-48g` : Substrate 8.0 GB, Tier-1 16.0 GB (8–10 lanes), Tier-2 24.0 GB (4–6 pipelines).
- **AC-07.1.1.3 (Floor Verification):** GIVEN a host with < 8 GB available RAM, THEN `profile.Validate` returns an error and refuses to start substrate.

CAP-08 **Dual-Constraint Dynamic Admission Controller**

REQ-04.2 · Package: `internal/admission`

Coordinates lane admissions using atomic file locking (`admission.lock`), enforcing concurrent count ceilings and cumulative RAM budgets simultaneously.

FEAT-08.1 **ATOMIC MEMORY BUDGET & SLOT ALLOCATION**

Manages `$AEP_STATE_DIR/admission.json` to reserve and release memory slots across both execution tiers.

TASK-08.1.1 **BUILD ATOMIC ADMISSION CONTROLLER** LLD T4

Implement file-locked reservation logic with automatic timeout expiration and garbage collection.

**Functions & Interfaces:**

```
type Controller struct {
  StateFile string
  LockFile string
}
func (c *Controller) Admit(ctx context.Context, req ReservationReq) (*Reservation, error)
func (c *Controller) Release(ctx context.Context, laneID string) error
func (c *Controller) CurrentState(ctx context.Context) (*AdmissionState, error)
```

**Acceptance Criteria (AC-08.1.1):**

- **AC-08.1.1.1 (Concurrency Safety):** GIVEN 10 parallel goroutines attempting to admit lanes when only 1 slot remains, THEN exactly one goroutine succeeds and 9 receive `ErrNoCapacity`; no race conditions occur.
- **AC-08.1.1.2 (Budget Exhaustion):** GIVEN a Tier-1 memory budget of 4.0 GB, WHEN two 2.0 GB lanes are active, THEN a 3rd lane request is rejected with `ErrNoCapacity` even if count ceiling is not reached.
- **AC-08.1.1.3 (Cgroup Enforcement):** GIVEN an admitted 2.0 GB lane, WHEN container runs with `--memory=2048m --memory-swap=2048m` and exceeds memory limit, THEN the container is OOM-killed (exit 137) without starving the substrate.

## Domain 5: REQ-05 Policy Enforcement & Platform Boot Gating

The platform must verify policy integrity against Git HEAD commits and enforce complete parity with legacy shell boot gates before initiating any container workloads.

## CAP-09 Cryptographic Policy Verifier & Environment Sanitizer

REQ-05.1 · Package: `internal/policy`

Guards `policy.data` against unauthorized uncommitted modifications and sanitizes execution environments.

### FEAT-09.1 GIT HEAD-BLOB TAMPER VERIFICATION

Reads `policy.data` from the working tree and compares its SHA-1/SHA-256 blob hash directly against `git rev-parse HEAD:policy.data`.

#### TASK-09.1.1 BUILD POLICY GUARD PACKAGE LLD T2

Implement tamper detection and environment variable scrubbers removing tokens from subprocesses.

**Functions & Interfaces:**

```
func VerifyAgainstHEAD(ctx context.Context, repoRoot string) (bool, error)
func ScrubEnv(env []string) []string
func ComputePolicyHash(data []byte) string
```

**Acceptance Criteria (AC-09.1.1):**

- **AC-09.1.1.1 (Tamper Lockout):** GIVEN `policy.data` modified on disk without committing to git, WHEN `VerifyAgainstHEAD` executes, THEN it returns `false` and causes `aep boot` to fail with exit 1.
- **AC-09.1.1.2 (Missing Policy Handling):** GIVEN `policy.data` does not exist, THEN the system emits `POLICY-INACTIVE` token, matching legacy `boot-gate.sh` behavior.
- **AC-09.1.1.3 (Token Scrubbing):** GIVEN an environment containing `GITHUB_TOKEN`, `PAT`, or `API_SECRET`, WHEN `ScrubEnv` runs, THEN all sensitive keys are purged from the resulting slice.

## CAP-10 Full-Parity Boot Gate (`aep boot`)

REQ-05.2 · Package: `cmd/aep` & `internal/boot`

Replaces `scripts/boot-gate.sh` with a Go-native implementation providing byte-identical validation tokens and substrate dependency checks.

### FEAT-10.1 SUBSTRATE READINESS & HEALTH PROBE ORCHESTRATION

Verifies Git cleanliness, NeuroStrata connectivity, container daemon status, and hardware profile before allowing supervisor dispatch.

#### TASK-10.1.1 IMPLEMENT NATIVE GO BOOT GATE LLD T3

Port shell logic to Go CLI with parity test harness validating identical output tokens.

**Functions & Interfaces:**

```
type GateRunner struct {
  RepoPath string
  Profile *profile.Profile
}
func (g *GateRunner) RunAllChecks(ctx context.Context) (*BootReport, error)
func DiffWithShellGate(ctx context.Context, stubEnv map[string]string) error
```

**Acceptance Criteria (AC-10.1.1):**

- **AC-10.1.1.1 (Token Parity):** GIVEN identical stub test environments, WHEN `aep boot` runs vs `boot-gate.sh`, THEN the emitted `BOOT-GATE:` lines match exactly in order and tokens.
- **AC-10.1.1.2 (Substrate Gate):** GIVEN container daemon or NeuroStrata unavailable, WHEN `aep boot` runs, THEN it exits 1 with a descriptive remediation string.

## Domain 6: REQ-06 Verification, Quarantine & Cryptographic Auditability

The platform must ensure that untrusted patches are never verified on the host kernel and that all task completions emit verifiable cryptographic receipts before ledger settlement.

### CAP-11 Disposable Sandbox Patch Re-Verification

`REQ-06.1 · Package: internal/verifier`

Runs untrusted repository test scripts ( `npm test` , `pytest` , `cargo test` ) inside fresh, disposable hardware microVMs or gVisor sandboxes.

#### FEAT-11.1 HOST KERNEL PROTECTION FROM UNTRUSTED PAYLOADS

Restricts Tier-1 verification to static analysis only; enforces dynamic execution inside disposable Tier-2 sandboxes.

#### TASK-11.1.1 BUILD DISPOSABLE SANDBOX VERIFIER RUNNER LLD T11

Construct test executor launching disposable microVMs with closed egress, applying patch, running build/test, and capturing cryptographic test logs.

**Functions & Interfaces:**

```
type Verifier interface {
  StaticCheck(ctx context.Context, patchPath string) (*StaticReport, error)
  DynamicVerifyInSandbox(ctx context.Context, patchPath string) (*VerifyResult, error)
}
```

**Acceptance Criteria (AC-11.1.1):**

- **AC-11.1.1.1 (No Untrusted Execution on Host):** GIVEN a patch generated by Tier 2 (ESF), WHEN dynamic tests run via `aep verify` , THEN tests execute inside a fresh disposable sandbox and NEVER directly on the host or Tier-1 container.
- **AC-11.1.1.2 (Disposable Lifecycle):** GIVEN test execution finishes (pass or fail), THEN the disposable microVM sandbox is destroyed immediately without leaving residual processes or disk state.

## CAP-12 Cryptographic Run Receipts & Ledger Settlement

REQ-06.2 · Package: `internal/receipt` & `internal/ledger`

Generates SHA-256 tail digests of all run outputs, binding git commit SHA, test logs, and memory inscriptions into a durable receipt.

### FEAT-12.1 AUDIT RECEIPT GENERATION & BEADS ISSUE CLOSING GATE

Enforces that an issue in Beads ( `bd close` ) can only transition to `closed` if accompanied by a cryptographically verified receipt.

#### TASK-12.1.1 IMPLEMENT RECEIPT PACKAGE & CLOSING GUARD

Create receipt generator computing Merkle/tail digests over run logs, git diffs, and test outputs.

**Functions & Interfaces:**

```
type Receipt struct {
  IssueID string `json:"issue_id"`
  CommitSHA string `json:"commit_sha"`
  PatchDigest string `json:"patch_digest"`
  EgressProof string `json:"egress_proof"`
  TailDigest string `json:"tail_digest"`
}
func Generate(run Record) (*Receipt, error)
func VerifyReceipt(r *Receipt) bool
func CloseWithReceipt(ctx context.Context, issueID string, r *Receipt) error
```

**Acceptance Criteria (AC-12.1.1):**

- **AC-12.1.1.1 (Tail Digest Verification):** GIVEN a completed task, WHEN `Generate` runs, THEN it computes SHA-256 over `commit_sha + patch_digest + egress_proof` .
- **AC-12.1.1.2 (Settlement Guard):** GIVEN an issue closure attempt, WHEN `CloseWithReceipt` executes, THEN it checks that the receipt signature is valid before executing `bd close <id>` .